XML: The Future That Arrived Sideways — A Retrospective from Hype Peak to the Age of AI

TL;DR

- XML did not become "the future" as a universal data layer, but it won decisively where it always belonged: documents and regulated enterprise systems. It powers Microsoft Office (.docx/.xlsx via OOXML/ISO 29500), SVG, RSS/Atom, SOAP/SAML, healthcare (HL7), finance (ISO 20022), and aerospace/defense (S1000D) — while losing the web-API and config wars to JSON and YAML.
- The surprising twist is that XML-style tagging has been reborn inside AI: Anthropic explicitly recommends wrapping prompt sections in XML tags (`<document>`, `<instructions>`), making 2004-era markup thinking newly relevant for LLM context engineering.
- On AIML: the user's "2014" date is a near-miss. AIML was created by Dr. Richard Wallace ~1995–2001 (AIML 1.0 spec, Aug 2001); the AIML 2.0 Working Draft and the Java reference interpreter program-ab both date to January 2013, with the draft last revised March 9, 2014 — which is likely what "2014" refers to. AIML is itself an XML dialect, and lives on in Kuki/Mitsuku, the five-time Loebner Prize winner.

Key Findings

1. XML's pedigree is impeccable and its lineage matters. XML 1.0 became a W3C Recommendation on February 10, 1998, distilled from SGML (ISO 8879:1986) by a working group chaired by Jon Bosak of Sun Microsystems, growing out of the SGML Editorial Review Board formed in 1996. Its design goals explicitly downgraded terseness ("Terseness in XML markup is of minimal importance" — goal #10) while prioritizing being "human-legible and reasonably clear" (goal #6) and easy to process (goal #4). This DNA — document-first, verbosity-tolerant — explains both its triumphs and its defeats. arxiv San Jose State University

2. The 2000–2004 hype was a genuine ecosystem, not just a format. Around the peak, XML spawned an entire technology stack: XSLT and XPath (W3C Recs, Nov 1999), XQuery, XML Schema (XSD), DTD, SOAP, WSDL, XHTML, RSS, RDF/RDF-XML, SVG, DocBook/DITA, XSL-FO, and XForms. The promise was platform independence, self-describing data, separation of content and presentation, and universal interoperability. Industry observers genuinely believed XML would be the substrate of everything.

3. The hype faded for concrete reasons. Verbosity, full-stack complexity (namespaces + schemas + transformation pipelines), parsing overhead, and poor developer ergonomics. The

AJAX revolution quietly became JSON-centric ("we still called it AJAX because AJAJ was too hard to pronounce"), REST displaced SOAP for web APIs, and YAML/TOML took over configuration. A widely-shared retrospective on the Bite Code! blog ("XML is the future," bitecode.dev/p/hype-cycles) captured the mood: "Turns out, XML was not the future. It was mostly technical debt. It was useful for things like documents, and I believe the most successful use of it are still MS Office and LibreOffice file formats. They are just zips of XML." Medium

4. Where XML genuinely won and still dominates:

- **Office documents:** Office Open XML (OOXML), standardized as ECMA-376 (Dec 2006) and ISO/IEC 29500 (Nov 2008), is the default format for .docx/.xlsx/.pptx — billions of files. Wikipedia
- **Graphics & syndication:** SVG and RSS 2.0/Atom remain fundamentally XML; podcast directories still require XML feeds. Progragon Technolabs
- **Enterprise/regulated:** SOAP and SAML 2.0 still run banking, government filing, and SSO; HL7 and ISO 20022 (finance) mandate XML; aerospace/defense uses S1000D and ATA iSpec 2200. Progragon Technolabs
- **Config in specific ecosystems:** Android layouts/manifests, Maven (pom.xml), Ant, MSBuild. Medium
- **Publishing:** DITA, DocBook, and JATS (scientific journals) depend on XSLT pipelines with no JSON equivalent. Progragon Technolabs

5. XML vs. modern formats in ML pipelines. For ML data interchange, JSON dominates, with YAML for config, Protocol Buffers/Avro for efficient binary serialization, and Parquet for columnar analytical storage. XML is generally absent from the modern ML data plane because its verbosity inflates both storage and token counts. Research comparing JSON and XML for LLM tasks finds JSON markedly superior in parseability: the ACL 2025 study "Evaluating Structured Output Robustness of Small Language Models" (arXiv:2507.01810) reports JSON outperforming XML on parseability across nearly all models (e.g., Qwen3-14B in the open setting scored JSON 98.1% vs. XML 43.0%; large 14B-class models averaged 90.3% parseability). XML retains an edge mainly for representing complex semantic hierarchies and mixed content.

6. The resurgence: tag-based structure inside LLMs. Anthropic's own documentation recommends XML tags as the primary structuring method for complex Claude prompts, using tags like `<instructions>`, `<context>`, `<example>`, `<document>`, and `<thinking>`. The mechanism is delimiting: tags create unambiguous boundaries between prompt sections. This is not full XML (no schemas, no validation, no namespaces) — it's XML's *syntactic skin* used as a delimiter system. Meanwhile, structured *output* is converging on JSON via

constrained/grammar-guided decoding (GBNF in llama.cpp, Outlines, XGrammar, vLLM's guided_json).

7. AIML — the early-2000s XML chatbot language, and the date correction. AIML (Artificial Intelligence Markup Language) is an XML dialect created by Dr. Richard S. Wallace and the Alicebot free-software community between roughly 1995 and 2001 for the A.L.I.C.E. chatbot (which "came to life" November 23, 1995). Per Wikipedia's biography of Wallace, "Wallace began work on A.L.I.C.E. in 1995, and the project has gained contributions from over 500 developers from around the world." AIML 1.0's spec was published August 2001. The user's "2014" almost certainly refers to the AIML 2.0 era: the AIML 2.0 Working Draft and its Java reference interpreter **program-ab** were both first released in January 2013, and the working draft was last revised **March 9, 2014** (Revision 1.0.2.22). So AIML was emphatically *not* created in 2014 — that's roughly when AIML 2.0 was being finalized as a draft. En Academic

Details

From SGML to the angle-bracket empire

XML is a restricted subset of SGML, the Standard Generalized Markup Language (ISO 8879:1986), itself descended from IBM's GML (Goldfarb, Mosher, Lorie, 1960s). The W3C's SGML Editorial Review Board (1996) became the XML Working Group under Jon Bosak; XML 1.0 was issued February 10, 1998 and is now in its Fifth Edition (2008). The genius of XML was formalizing *well-formedness* (parseable without a schema) as distinct from *validity* (conforming to a DTD/XSD), enabling lightweight processing. W3C + 4

The ecosystem matured fast: Namespaces in XML (Jan 1999), XSLT 1.0 + XPath 1.0 (Nov 1999), XML Schema (2001), then SOAP/WSDL for web services, XHTML, SVG, RSS/RDF, and document standards like DocBook and DITA. By 2004 (XML 1.1 and the XML 1.0 Third Edition both published Feb 4, 2004), the conventional wisdom was that XML would underlie all data exchange. W3C

Advantages that were real

XML's strengths are genuine and durable: a self-describing/self-documenting structure; rich schema validation (DTD, XSD, RelaxNG, Schematron); namespaces for vocabulary mixing; first-class **mixed content** (text interleaved with inline markup) that JSON cannot represent naturally; mature transformation (XSLT) and querying (XPath/XQuery); and exceptional standardization and stability. For *document-centric* data — where a paragraph contains bold runs, footnotes, and cross-references inline — XML remains the correct tool, which is exactly why OOXML, DITA, S1000D, and JATS are XML and not JSON.

Disadvantages that killed the hype

The verbosity that goal #10 declared unimportant turned out to matter enormously when XML was misapplied as a wire format and config language. Every element needs a closing tag, roughly doubling payload size versus JSON; SOAP's WSDL contracts created tight coupling; the full stack (XSD + XSLT + namespaces + SOAP envelopes) imposed a steep learning curve. As JavaScript ate the web, JSON's direct mapping to JS objects (JSON.parse/stringify) made XML feel like ceremony. For configuration, YAML (created 2001, "YAML Ain't Markup Language") and later TOML won on human-readability and comment support. Even Maven, the archetypal XML build tool, spawned a "Polyglot Maven" project explicitly "to escape from XML." Raygun Baeldung

XML in the age of ML and AI

As a data format for ML: XML barely appears in modern ML pipelines. JSON is the lingua franca for training-data records, information extraction, and function-calling I/O; YAML configures experiments; Protocol Buffers and Avro handle efficient serialization; Parquet stores columnar feature data. The reason is structural: token-priced LLM APIs and bandwidth-sensitive pipelines punish XML's closing-tag overhead. Benchmarks consistently show XML consuming fewer *input* tokens than pretty-printed JSON for hierarchical structures in some cases but producing far more verbose *output*, and being more error-prone for LLMs to emit exactly (strict closing-tag requirements). arxiv

As prompt structure (the comeback): Here XML markup thinking has had a genuine renaissance. Anthropic's prompt-engineering guidance is explicit: "When your prompts involve multiple components like context, instructions, and examples, XML tags can be a game-changer." Claude is documented as responding especially well to tags like `<document>`, `<document_content>`, `<source>`, `<instructions>`, `<example>`, and `<thinking>`. Crucially, the value is *delimiting*, not validation — there are "no canonical 'best' XML tags that Claude has been trained with," only the principle of clear, consistent, nestable boundaries. This is the deep irony: in 2004 XML promised machine-readable structure for programs; in 2025 its tag syntax provides *human-and-model-readable* structure for neural networks that have no XML parser at all — they simply learned that angle-bracket delimiters mark boundaries. Mintlify

Structured generation: For guaranteed-valid structured *output*, the industry standardized on JSON Schema + constrained decoding (token masking against a grammar), implemented via GBNF (llama.cpp), Outlines, XGrammar, and vLLM/SGLang. XML is a possible grammar target but not the default. Note also a documented tension: forcing strict format constraints can degrade reasoning quality. The "Let Me Speak Freely?" line of work and subsequent benchmarking found that math, symbolic, and complex-analysis tasks showed roughly 10–15%

performance degradation when models were locked into JSON-mode versus free-form generation — which is why practitioners often separate a free reasoning pass from a structured formatting pass. Medium

RAG and document AI: The dominant intermediate representation for feeding documents to LLMs is now **Markdown**, not XML — because it preserves structure (headings, tables, lists) with minimal token overhead and embeds predictably. Microsoft's MarkItDown converts Office/PDF formats to Markdown for LLM pipelines. The twist: those very Office formats *are* XML (OOXML) under the hood, so XML remains the durable source-of-truth that gets *down-converted* to Markdown for model consumption. DEV Community AI Builder Club

AIML: rule-based "AI" in XML clothing

AIML is itself an XML-derived markup language. Its core constructs are XML elements:

- `<aiml>` — root element
- `<category>` — the atomic unit of knowledge (one stimulus-response rule)
- `<pattern>` — the input to match (with wildcards `*` and `_`, plus AIML 2.0's `^`, `#`, `$` priority)
- `<template>` — the response
- `<srai>` — recursive "symbolic reduction" that re-feeds transformed input through the matcher (with shorthand `<sr/>` = `<srai><star/></srai>`) Aiml
- `<star/>` — captures wildcard matches
- `<that>` — matches the bot's previous utterance (context)
- `<topic>`, `<think>`, `<set>`/`<get>`, `<random>`/`<li>`, `<condition>` — control flow and state

A minimal example:

```xml
```

```
<?xml version="1.0" encoding="UTF-8"?>
<aiml version="2.0">
  <category>
    <pattern>WHO IS ALBERT EINSTEIN</pattern>
    <template>Albert Einstein was a German physicist.</template>
  </category>
  <category>
    <pattern>DO YOU KNOW WHO * IS</pattern>
    <template><srai>WHO IS <star/></srai></template>
  </category>
</aiml>
```

The historical timeline (correcting "2014"):

- **Nov 23, 1995:** A.L.I.C.E. ("Artificial Linguistic Internet Computer Entity") created by Richard Wallace; named after the computer it first ran on.
- **1995–2000/2001:** AIML developed by Wallace and the Alicebot open-source community (eventually 500+ contributors worldwide); rewritten in Java from 1998.
- **Aug 3, 2001:** AIML 1.0 specification published. A.L.I.C.E. won the Loebner Prize in 2000, 2001, and 2004. (En Academic) (arxiv)
- **2001–2002:** Annotated A.L.I.C.E. (AAA) AIML set released under GNU GPL, enabling clones.
- **January 2013:** AIML 2.0 Working Draft released; **program-ab**, the Java reference implementation, first released by Richard Wallace (info@alicebot.org).
- **March 9, 2014:** AIML 2.0 Working Draft last revised (Revision 1.0.2.22) — the most likely source of the "2014" association. (GitHub)

AIML 2.0 added Sets, Maps, local variables, loops, `<learn>`/`<learnf>` (runtime learning), `<sraix>` (calling external web services and other bots), and OOB (out-of-band) tags for device control. (GitHub)

The irony and the contrast. AIML was an early-2000s attempt at conversational "AI" built entirely on XML markup and hand-authored pattern-matching — supervised learning where a human "botmaster" writes categories. A.L.I.C.E. "consists of roughly 41,000 elements called categories which is the basic unit of knowledge in AIML" (Galitsky et al., arXiv:1705.01214), versus ELIZA's ~200. This is the antithesis of modern transformer LLMs: AIML is deterministic, rule-based, interpretable, and incapable of generalization beyond its hand-written rules; LLMs are probabilistic, learned-from-data, opaque, and generative. Yet AIML's `<srai>` recursion and `<that>` context tracking prefigure ideas — input normalization, intent reduction, conversational

state — that reappear in modern dialogue systems. AIML 2.1 even renamed its "simplest canonical form" / "terminal category" to "Intent," explicitly aligning with the vocabulary of modern NLU platforms. google

Is AIML still used? Yes, in niches. The standout is **Kuki** (formerly **Mitsuku**), created by **Steve Worswick** (~2005) on Pandorabots' AIML technology. Mitsuku/Kuki won the Loebner Prize a record **five times (2013, 2016, 2017, 2018, 2019)**, earning Worswick a Guinness World Record. Kuki is hand-authored AIML at remarkable scale — over 350,000 hand-crafted patterns by 2019 — and is engagement-oriented: per its own research page, "Kuki averages 64 Conversation-turns Per Session (CPS), which is 3x higher than Microsoft XiaoIce... and 8x higher than is industry standard," and it has exchanged over one billion messages with an estimated 25 million human end-users. It is now positioned for brand engagement and the metaverse (it has modeled NFTs for Vogue and run campaigns for H&M). Pandorabots remains the primary commercial AIML platform. Grokipedia Wikipedia

Working URLs (source code, specs, authorities)

W3C specifications:

- XML 1.0 (Fifth Edition): https://www.w3.org/TR/xml/
- Original XML 1.0 Recommendation (1998): https://www.w3.org/TR/1998/REC-xml-19980210
- XML 1.0 design goals press release (1998): https://www.w3.org/press-releases/1998/xml10-rec/

Office Open XML / standards:

- ECMA-376: https://ecma-international.org/publications-and-standards/standards/ecma-376/
- ISO/IEC 29500 (Library of Congress format description): https://www.loc.gov/preservation/digital/formats/fdd/fdd000395.shtml

AIML — specifications:

- AIML Foundation spec page: http://www.aiml.foundation/doc.html
- AIML 2.0 Working Draft (Google Docs, referenced in program-ab source; Rev 1.0.2.22, Mar 9, 2014): https://docs.google.com/document/d/1wNT25hJRyupcG51aO89UcQEiG-HkXRXusukADpFnDs4/pub
- AIML 2.0 Spec on GitHub: https://github.com/AIML-Foundation/AIML-2.0-Spec
- Richard Wallace's spec fork: https://github.com/drwallace/AIML-Spec

- AIML 1.0.1 (alicebot.org): http://www.alicebot.org/TR/2005/WD-aiml/
- Pandorabots AIML basics: https://www.pandorabots.com/docs/aiml-basics/
- Pandorabots AIML fundamentals/reference: https://pandorabots.com/docs/aiml-fundamentals/ and https://www.pandorabots.com/docs/aiml-reference/

AIML — interpreters / source code:

- Dr. Richard Wallace's GitHub: https://github.com/drwallace
  - Free ALICE AIML brain: https://github.com/drwallace/aiml-en-us-foundation-alice
- program-ab (Java reference interpreter) — original Google Code archive: https://code.google.com/archive/p/program-ab/
  - Canonical GitHub mirror: https://github.com/lumenrobot/program-ab (also https://github.com/miguelff/program-ab)
- program-o (PHP) — archived: https://github.com/Program-O/Program-O
  - Successor "Lemur Engine": https://github.com/theramenrobotdiscocode/lemur-engine and https://github.com/lemurengine/lemurbot
- program-y (Python, Keith Sterling): https://github.com/keiffster/program-y — PyPI: https://pypi.org/project/programy/
- PyAIML (original, Cort Stratton): https://github.com/cdwfs/pyaiml
- python-aiml (maintained fork, Paulo Villegas): https://github.com/paulovn/python-aiml — PyPI: https://pypi.org/project/python-aiml/

AIML — platforms / bots:

- Pandorabots: https://www.pandorabots.com/
- Kuki/Mitsuku: https://www.kuki.ai/ (research: https://www.kuki.ai/research)

Recommendations

If you are choosing a data format today:

- **Use JSON** for web APIs, ML data records, and inter-service messaging — it's the default for good reasons. Add JSON Schema + constrained decoding when you need guaranteed-valid LLM output.
- **Use YAML/TOML** for human-edited configuration (comments, readability).
- **Use Parquet** for columnar/analytical ML data at scale; **Protobuf/Avro** for high-throughput binary serialization.
- **Use XML when — and only when — you have document-centric mixed content, must validate against a legally-mandated schema, or must integrate with an entrenched XML**

ecosystem (Office formats, HL7/FHIR-XML, ISO 20022, SAML, S1000D, DITA/DocBook publishing). Don't fight these; XML is genuinely the right tool there.

If you are doing LLM prompt/context engineering:

- **Adopt XML-style tags for prompt structure** (`<instructions>`, `<context>`, `<document>`, `<examples>`, `<thinking>`), especially with Claude. Keep tag names descriptive and consistent; nest for hierarchy. This is the highest-ROI place XML thinking pays off in 2025.
- **Do not use XML as the bulk encoding** for large repeated tabular payloads in prompts — it wastes tokens. Use it for *boundaries*, JSON/CSV/Markdown for *content*.
- **For document RAG, convert XML/OOXML sources to Markdown** (e.g., MarkItDown/Docling) for ingestion, while keeping the XML as the structured source of truth.

If you are working with AIML or conversational rule systems:

- For learning/experimentation, use **program-y** (Python, actively maintained, AIML 2.1) or **python-aiml**; for hosted bots, **Pandorabots**.
- Treat AIML as a *complement* to LLMs, not a competitor: rule-based pattern matching gives deterministic, auditable answers for safety-critical or fixed-response paths (compliance, disclaimers, escalation), which you can hybridize with an LLM for open-domain conversation.

Benchmarks/thresholds that would change these recommendations: If a token-efficient structured format (e.g., TOON or evidence-alias schemes) demonstrates both major token savings *and* schema-valid reliability across frontier models, prefer it over JSON for high-volume tabular prompt input. If constrained-decoding overhead drops to near-zero (already largely true on vLLM/SGLang with XGrammar) and reasoning-degradation concerns are resolved, default to constrained JSON output everywhere.

Caveats

- **"XML was the future" — the verdict is split, not binary.** It's partial vindication (XML is genuinely everywhere in documents and regulated enterprise, and quietly underlies Office files most people open daily) and partial failure (it lost the web-API and config wars and is largely absent from the ML data plane). Both halves are true simultaneously.
- **The LLM/XML-tag connection is real but easy to overstate.** Models don't parse XML; they pattern-match on delimiters they saw in training. XML tags are *a* good delimiter convention, not a magic key, and other delimiter schemes (Markdown headers, custom tokens like `<|...|>`) also work. Vendor claims of "20–40% more consistent outputs" from XML structuring come from interested parties (prompt-tooling blogs and Anthropic's own

docs) and should be read as directional, not as independently audited benchmarks. AI Prompt Library

- **AIML dates carry minor source disagreement.** Sources variously give the AIML/ALICE origin as 1995, 1998 (Java rewrite), or 2001 (AIML 1.0 spec). The defensible framing: conceived 1995, AIML formalized 1995–2001, AIML 1.0 spec August 2001, AIML 2.0 draft + program-ab January 2013, draft last revised March 2014. The user's "2014" maps to the AIML 2.0 draft finalization, not AIML's creation.
- **Some cited comparison numbers are illustrative.** Token-count and accuracy comparisons between JSON/XML/YAML/TOON vary by model, tokenizer, and task; treat specific percentages as snapshots rather than universal constants.
- **AIML repository canonicality is fuzzy.** program-ab's true original is the now-archived Google Code project; the GitHub "mirrors" are community forks, not an official Wallace-owned repository. program-o is archived in favor of the Lemur Engine. GitHub